

习题三

一、选择题

- 对于栈操作数据的原则是 ()。
A. 先进先出 B. 后进先出 C. 后进后出 D. 不分顺序
- 一个栈的输入序列为 $123\cdots n$, 若输出序列的第一个元素是 n , 输出第 i ($1 \leq i \leq n$) 个元素是 ()。
A. 不确定 B. $n-i+1$ C. i D. $n-i$
- 若一个栈的输入序列为 $1,2,3,\cdots,n$, 输出序列的第一个元素是 i , 则第 j 个输出元素是 ()。
A. $i-j-1$ B. $i-j$ C. $j-i+1$ D. 不确定的
- 若已知一个栈的入栈序列是 $1,2,3,\cdots,n$, 其输出序列为 $p_1,p_2,p_3,\cdots,p_N$, 若 p_N 是 n , 则 p_i 是 ()。
A. i B. $n-i$ C. $n-i+1$ D. 不确定
- 有六个元素 $6, 5, 4, 3, 2, 1$ 的顺序进栈, 问下列哪一个不是合法的出栈序列? ()
A. $5\ 4\ 3\ 6\ 1\ 2$ B. $4\ 5\ 3\ 1\ 2\ 6$ C. $3\ 4\ 6\ 5\ 2\ 1$ D. $2\ 3\ 4\ 1\ 5\ 6$
- 输入序列为 ABC, 可以变为 CBA 时, 经过的栈操作为 ()
A. push,pop,push,pop,push,pop B. push,push,push,pop,pop,pop
C. push,push,pop,pop,push,pop D. push,pop,push,push,pop,pop
- 若一个栈以向量 $V[1..n]$ 存储, 初始栈顶指针 top 为 $n+1$, 则下面 x 进栈的正确操作是 ()。
A. $top=top+1; V[top]=x$ B. $V[top]=x; top=top+1$
C. $top=top-1; V[top]=x$ D. $V[top]=x; top=top-1$
- 若栈采用顺序存储方式存储, 现两栈共享空间 $V[1..m]$, $top[i]$ 代表第 i 个栈 ($i=1,2$) 栈顶, 栈 1 的底在 $v[1]$, 栈 2 的底在 $V[m]$, 则栈满的条件是 ()。
A. $|top[2]-top[1]|=0$ B. $top[1]+1=top[2]$ C. $top[1]+top[2]=m$ D. $top[1]=top[2]$
- 栈在 () 中应用。
A. 递归调用 B. 子程序调用 C. 表达式求值 D. A, B, C
- 一个递归算法必须包括 ()。
A. 递归部分 B. 终止条件和递归部分 C. 迭代部分 D. 终止条件和迭代部分
- 执行完下列语句段后, i 值为: ()

```
int f(int x)
{ return ((x>0)? x*f(x-1):2); }
int i ;
i=f(f(1));
```


A. 2 B. 4 C. 8 D. 无限递归
- 表达式 $a*(b+c)-d$ 的后缀表达式是 ()。
A. $abcd*+-$ B. $abc*+d-$ C. $abc*+d-$ D. $-+*abcd$
- 表达式 $3*2^{(4+2*2-6*3)}-5$ 求值过程中当扫描到 6 时, 对象栈和算符栈为 (), 其中 $^$ 为乘幂。
A. $3,2,4,1,1; \#*^{(+*-}$ B. $3,2,8; \#*^{(-$ C. $3,2,4,2,2; \#*^{(-$ D. $3,2,8; \#*^{(-$
- 设计一个判别表达式中左, 右括号是否配对出现的算法, 采用 () 数据结构最佳。
A. 线性表的顺序存储结构 B. 队列 C. 线性表的链式存储结构 D. 栈
- 假设以数组 $A[m]$ 存放循环队列的元素, 其头尾指针分别为 $front$ 和 $rear$, 则当前队列中的元素个数为 ()。
A. $(rear-front+m)\%m$ B. $rear-front+1$ C. $(front-rear+m)\%m$ D. $(rear-front)\%m$
- 循环队列存储在数组 $A[0..m]$ 中, 则入队时的操作为 ()。
A. $rear=rear+1$ B. $rear=(rear+1) \bmod (m-1)$ C. $rear=(rear+1) \bmod m$ D. $rear=(rear+1) \bmod (m+1)$
- 若用一个大小为 6 的数组来实现循环队列, 且当前 $rear$ 和 $front$ 的值分别为 0 和 3, 当从队列中删除一个元素, 再加入两个元素后, $rear$ 和 $front$ 的值分别为多少? ()
A. 1 和 5 B. 2 和 4 C. 4 和 2 D. 5 和 1
- 设栈 S 和队列 Q 的初始状态为空, 元素 e_1, e_2, e_3, e_4, e_5 和 e_6 依次通过栈 S , 一个元素出栈后即进队列 Q , 若 6 个元素出队的序列是 $e_2, e_4, e_3, e_6, e_5, e_1$ 则栈 S 的容量至少应该是 ()。

A . 6 B. 4 C. 3 D. 2

19. 用单链表表示的链式队列的队头在链表的 () 位置。

A . 链头 B . 链尾 C . 链中

二、判断题

1. 消除递归不一定需要使用栈, 此说法 ()
2. 栈是实现过程和函数等子程序所必需的结构。()
3. 两个栈共用静态存储空间, 对头使用也存在空间溢出问题。()
4. 两个栈共享一片连续内存空间时, 为提高内存利用率, 减少溢出机会, 应把两个栈的栈底分别设在这片内存空间的两端。()
5. 即使对不含相同元素的同一输入序列进行两组不同的合法的入栈和出栈组合操作, 所得的输出序列也一定相同。()
6. 有 n 个数顺序 (依次) 进栈, 出栈序列有 C_n 种, $C_n = [1/(n+1)] * (2n)! / [(n!)*(n!)]$ 。()
7. 栈与队列是一种特殊操作的线性表。()
8. 若输入序列为 1,2,3,4,5,6,则通过一个栈可以输出序列 3,2,5,6,4,1. ()
9. 任何一个递归过程都可以转换成非递归过程。()
10. 只有那种使用了局部变量的递归过程在转换成非递归过程时才必须使用栈。()
11. 队列逻辑上是一个下端和上端既能增加又能减少的线性表。()
12. 循环队列通常用指针来实现队列的头尾相接。()
13. 循环队列也存在空间溢出问题。()
14. 栈和队列的存储方式, 既可以是顺序方式, 又可以是链式方式。()

三、填空题

1. 栈是_____的线性表, 其运算遵循_____的原则。
2. _____是限定仅在表尾进行插入或删除操作的线性表。
3. 一个栈的输入序列是: 1, 2, 3 则不可能的栈输出序列是_____。
4. 设有一个空栈, 栈顶指针为 1000H(十六进制), 现有输入序列为 1, 2, 3, 4, 5, 经过 PUSH,PUSH,POP,PUSH,POP,PUSH,PUSH 之后, 输出序列是_____, 而栈顶指针值是_____H。设栈为顺序栈, 每个元素占 4 个字节。
5. 当两个栈共享一存储区时, 栈利用一维数组 $stack(1,n)$ 表示, 两栈顶指针为 $top[1]$ 与 $top[2]$, 则当栈 1 空时, $top[1]$ 为_____, 栈 2 空时, $top[2]$ 为_____, 栈满时为_____。
6. 在作进栈运算时应先判别栈是否 (1); 在作退栈运算时应先判别栈是否 (2); 当栈中元素为 n 个, 作进栈运算时发生上溢, 则说明该栈的最大容量为 (3)。为了增加内存空间的利用率和减少溢出的可能性, 由两个栈共享一片连续的空间时, 应将两栈的 (4) 分别设在内存空间的两端, 这样只有当 (5) 时才产生溢出。
7. 多个栈共存时, 最好用_____作为存储结构。
8. 用 S 表示入栈操作, X 表示出栈操作, 若元素入栈的顺序为 1234, 为了得到 1342 出栈顺序, 相应的 S 和 X 的操作串为_____。
9. 顺序栈用 $data[1..n]$ 存储数据, 栈顶指针是 top , 则值为 x 的元素入栈的操作是_____。
10. 表达式 $23 + ((12 * 3 - 2) / 4 + 34 * 5 / 7) + 108 / 9$ 的后缀表达式是_____。
11. 循环队列的引入, 目的是为了克服_____。
12. 用下标 0 开始的 N 元数组实现循环队列时, 为实现下标变量 M 加 1 后在数组有效下标范围内循环, 可采用的表达式是 $M =$ _____。
13. 队列的特点是_____。
14. 已知链队列的头尾指针分别是 f 和 r , 则将值 x 入队的操作序列是_____。

四、应用题

1. 假设以 S 和 X 分别表示入栈和出栈操作, 则对初态和终态均为空的栈操作可由 S 和 X 组成的序列表示 (如 SXSX)。
(1) 试指出判别给定序列是否合法的一般规则。

- (2) 两个不同合法序列（对同一输入序列）能否得到相同的输出元素序列？如能得到，请举例说明。
2. 有 5 个元素，其入栈次序为：A, B, C, D, E，在各种可能的出栈次序中，以元素 C, D 最先出栈（即 C 第一个且 D 第二个出栈）的次序有哪几个？
3. 设输入序列为 2, 3, 4, 5, 6，利用一个栈能得到序列 2, 5, 3, 4, 6 吗？栈可以用单链表实现吗？
4. 试证明：若借助栈由输入序列 $1, 2, \dots, n$ 得到输出序列为 $P_1, P_2, \dots, P_n$ （它是输入序列的一个排列），则在输出序列中不可能出现这样的情形：存在着 $i < j < k$, 使 $P_k < P_i < P_j$ 。
5. 用一个数组 S（设大小为 MAX）作为两个堆栈的共享空间。请说明共享方法，栈满/栈空的判断条件，并用 C 或 PASCAL 设计公用的入栈操作 $\text{push}(i, x)$ ，其中 i 为 0 或 1，用于表示栈号， x 为入栈值。
6. 画出对算术表达式 $A-B \times C/D-E \uparrow F$ 求值时操作数栈和运算符栈的变化过程。
7. 内存中一片连续空间（不妨假设地址从 1 到 m ）提供给两个栈 S1 和 S2 使用，怎样分配这部分存储空间，使得对任一个栈，仅当这部分空间全满时才发生上溢。
8. 在一个算法中需要建立多个堆栈时可以选用下列三种方案之一，试问：这三种方案之间相比较各有什么优缺点？
- (1) 分别用多个顺序存储空间建立多个独立的堆栈；
 - (2) 多个堆栈共享一个顺序存储空间；
 - (3) 分别建立多个独立的链接堆栈。

五、算法设计题

1. 设从键盘输入一整数的序列： $a_1, a_2, a_3, \dots, a_n$ ，试编写算法实现：用栈结构存储输入的整数，当 $a_i \neq -1$ 时，将 a_i 进栈；当 $a_i = -1$ 时，输出栈顶整数并出栈。算法应对异常情况（入栈满等）给出相应的信息。
2. 设表达式以字符形式已存入数组 $E[n]$ 中，'#' 为表达式的结束符，试写出判断表达式中括号（'（' 和 '）'）是否配对的 C 语言描述算法：EXYX(E); (注：算法中可调用栈操作的基本算法。)
3. 假设以 I 和 O 分别表示入栈和出栈操作。栈的初态和终态均为空，入栈和出栈的操作序列可表示为仅由 I 和 O 组成的序列，称可以操作的序列为合法序列，否则称为非法序列。
- (1) 下面所示的序列中哪些是合法的？
A. IOIOIOO B. IOOIOIOO C. IIIIOIOIO D. IIIOOIOO
 - (2) 通过对 (1) 的分析，写出一个算法，判定所给的操作序列是否合法。若合法，返回 true，否则返回 false（假定被判定的操作序列已存入一维数组中）。
4. 设整数序列 $a_1, a_2, \dots, a_n$ ，给出求解最大值的递归程序。
5. 设计算法以求解从集合 $\{1..n\}$ 中选取 k ($k \leq n$) 个元素的所有组合。例如，从集合 $\{1..4\}$ 中选取 2 个元素的所有组合的输出结果为：1 2, 1 3, 1 4, 2 3, 2 4, 3 4。

参考答案：

一、选择题

1	B	2	B	3	D	4	D	5	C	6	B	7	C	8	B	9	D	10	B
11	B	12	B	13	D	14	D	15	A	16	D	17	B	18	C	19	A		

二、判断题

1	√	2	√	3	√	4	√	5	×	6	√	7	√
8	√	9	√	10	×	11	√	12	×	13	√	14	√

三、填空

三 填空题

1. 操作受限（或限定仅在表尾进行插入和删除操作） 后进先出	2. 栈
3. 3 1 2	4. 23 100CH
5. 0 $n+1$ $\text{top}[1]+1=\text{top}[2]$	6. (1)满 (2)空 (3) n (4)栈底 (5)两栈顶指针相邻（即值之差的绝对值为 1）

7.链式存储结构	8 . $S \times SS \times S \times \times$
9. <code>data[++top]=x;</code>	10 . $23.12.3*2-4/34.5*7/++108.9/+$ (注: 表达式中的点(.)表示将数隔开, 如 23.12.3 是三个数)
11.假溢出时大量移动数据元素。	12 . $(M+1)\% N;$
13.先进先出	14. <code>s=(LinkedList)malloc(sizeof(LNode)) ;</code> <code>s->data=x;s->next=r->next; r->next=s; r=s;</code>

四 应用题

1. (1) 通常有两条规则。第一是给定序列中 S 的个数和 X 的个数相等; 第二是从给定序列的开始, 到给定序列中的任一位置, S 的个数要大于或等于 X 的个数。

(2) 可以得到相同的输出元素序列。例如, 输入元素为 A, B, C, 则两个输入的合法序列 ABC 和 BAC 均可得到输出元素序列 ABC。对于合法序列 ABC, 我们使用本题约定的 $S \times S \times S$ 操作序列; 对于合法序列 BAC, 我们使用 $SS \times \times S$ 操作序列。

2.三个: CDEBA, CDBEA, CDBAE

3.不能得到序列 2, 5, 3, 4, 6。栈可以用单链表实现, 这就是链栈。由于栈只在栈顶操作, 所以链栈通常不设头结点。

4.如果 $i < j$, 则对于 $p_i < p_j$ 情况, 说明 p_i 在 p_j 入栈前先出栈。而对于 $p_i > p_j$ 的情况, 则说明要将 p_i 压到 p_j 之上, 也就是在 p_j 出栈之后 p_i 才能出栈。这就说明, 对于 $i < j < k$, 不可能出现 $p_i < p_k < p_j$ 的输出序列。换句话说, 对于输入序列 1, 2, 3, 不可能出现 3, 1, 2 的输出序列。

5.两栈共享一向量空间 (一维数组), 栈底设在数组的两端, 两栈顶相邻时为栈满。设共享数组为 $S[\text{MAX}]$, 则一个栈顶指针为 -1, 另一个栈顶指针为 MAX 时, 栈为空。

用 C 写的入栈操作 `push (i, x)` 如下:

`const MAX=共享栈可能达到的最大容量`

`typedef struct node`

`{elemtype s[MAX];`

`int top[2];`

`}anode;`

`anode ds;`

`int push(int i,elemtype x)`

//ds 为容量有 MAX 个类型为 elemtype 的元素的一维数组, 由两个栈共享其空间。i 的值为 0 或 1, x 为类型为 elemtype 的元素。本算法将 x 压入栈中。如压栈成功, 返回 1; 否则, 返回 0。

`{if (ds.top[1]-ds.top[0]==1) {printf ("栈满\n"); return (0); }`

`switch (i)`

`{case 0: ds.s[++ds.top[i]]=x; break;`

`case 1: ds.s[--ds.top[i]]=x;`

`return (1); }//入栈成功。`

`}`

6. 设操作数栈是 opnd, 操作符栈是 optr, 对算术表达式 $A-B \times C/D-E \uparrow F$ 求值, 过程如下:

步骤	opnd 栈	optr 栈	输入字符	主要操作
初始		#	$A-B \times C/D-E \uparrow F \#$	PUSH(OPTR,'#')
1	A	#	$\underline{A}-B \times C/D-E \uparrow F \#$	PUSH(OPND,A)
2	A	# -	$\underline{-}B \times C/D-E \uparrow F \#$	PUSH(OPTR,'-')
3	AB	# -	$\underline{B} \times C/D-E \uparrow F \#$	PUSH(OPND,B)
4	AB	# - *	$\underline{*}C/D-E \uparrow F \#$	PUSH(OPTR,'*')
5	ABC	# - *	$\underline{C}/D-E \uparrow F \#$	PUSH(OPND,C)

6	AT(T=B*C)	# - /	$\sqrt{D-E} \uparrow F\#$	PUSH(OPND,POP(OPND)*POP(OPND)) PUSH(OPTR,'/')
7	ATD	# - /	$\sqrt{D-E} \uparrow F\#$	PUSH(OPND,D)
8	AT(T=T/D) T(T=A-T)	# - # -	$\sqrt{E} \uparrow F\#$	x=POP(OPND);y=POP(OPND) PUSH(OPND,y/x); x=POP(OPND);y=POP(OPND); PUSH(OPND,y-x) PUSH(OPTR,'-')
9	TE	# -	$\sqrt{E} \uparrow F\#$	PUSH(OPND,E)
10	TE	# - $\uparrow$	$\uparrow F\#$	PUSH(OPTR, ' $\uparrow$ ')
11	TEF	# - $\uparrow$	$F\#$	PUSH(OPND,F)
12	TE TS(S=E $\uparrow$ F) R(R=T-S)	#- #	$\#$	X=POP(OPND) Y=POP(OPND) POP(OPTR) PUSH(OPND,y $\uparrow$ x) x=POP(OPND) y=POP(OPND) POP(OPTR) PUSH(OPND,y-x)

7. S1 和 S2 共享内存中一片连续空间（地址 1 到 m），可以将 S1 和 S2 的栈底设在两端，两栈顶向共享空间的中心延伸，仅当两栈顶指针相邻（两栈顶指针值之差的绝对值等于 1）时，判断为栈满，当一个栈顶指针为 0，另一个栈顶指针 m+1 时为两栈均空。

8. （1）每个栈仅用一个顺序存储空间时，操作简便，但分配存储空间小了，容易产生溢出，分配空间大了，容易造成浪费，各栈不能共享空间。

（2）多个栈共享一个顺序存储空间，充分利用了存储空间，只有在整个存储空间都用完时才能产生溢出，其缺点是当一个栈满时要向左、右栈查询有无空闲单元。如果有，则要移动元素和修改相关的栈底和栈顶指针。当接近栈满时，查询空闲单元、移动元素和修改栈底栈顶指针的操作频繁，计算复杂并且耗费时间。

（3）多个链栈一般不考虑栈的溢出（仅受用户内存空间限制），缺点是栈中元素要以指针相链接，比顺序存储多占用了存储空间。

五 算法设计题

1. #define maxsize 栈空间容量

```
void InOutS(int s[maxsize])
```

//s 是元素为整数的栈，本算法进行入栈和退栈操作。

```
{int top=0; //top 为栈顶指针，定义 top=0 时为栈空。
```

```
for(i=1; i<=n; i++) //n 个整数序列作处理。
```

```
{scanf("%d",&x); //从键盘读入整数序列。
```

```
if(x!=-1) // 读入的整数不等于-1 时入栈。
```

```
if(top==maxsize-1){printf("栈满\n");exit(0);}else s[++top]=x; //x 入栈。
```

```
else //读入的整数等于-1 时退栈。
```

```
{if(top==0){printf("栈空\n");exit(0);} else printf("出栈元素是%d\n",s[top--]); }}
```

```
//算法结束。
```

2.[题目分析]判断表达式中括号是否匹配，可通过栈，简单说是左括号时进栈，右括号时退栈。退栈时，若栈顶元素是左括号，则新读入的右括号与栈顶左括号就可消去。如此下去，输入表达式结束时，栈为空则正确，否则括号不匹配。

```
int EXYX(char E[],int n)
```

//E[]是有 n 字符的字符数组，存放字符串表达式，以 '#' 结束。本算法判断表达式中圆括号是否匹配。

```
{char s[30]; //s 是一维数组，容量足够大，用作存放括号的栈。
```

```
int top=0; //top 用作栈顶指针。
```

```
s[top]='#'; //'#' 先入栈，用于和表达式结束符号 '#' 匹配。
```

```
int i=0; //字符数组 E 的工作指针。
```

```

while(E[i]!='#') //逐字符处理字符表达式的数组。
switch(E[i])
{case'(': s[++top]='('; i++; break;
case')': if(s[top]=='('){top--; i++; break;}
          else{printf("括号不配对");exit(0);}
case'#': if(s[top]=='#{printf("括号配对\n");return (1);}
          else {printf(" 括号不配对\n");return (0);} //括号不配对
default: i++; //读入其它字符，不作处理。 }
} //算法结束。

```

[算法讨论]本题是用栈判断括号匹配的特例：只检查圆括号的配对。一般情况是检查花括号（{，}）、方括号（[，]）和圆括号（（，））的配对问题。编写算法中如遇左括号（{，[，或（）就压入栈中，如遇右括号（}，]，或）），则与栈顶元素比较，如是与其配对的括号（左花括号，左方括号或左圆括号），则弹出栈顶元素；否则，就结论括号不配对。在读入表达式结束符'#'时，栈中若应只剩'#'，表示括号全部配对成功；否则表示括号不匹配。

另外，由于本题只是检查括号是否匹配，故对从表达式中读入的不是括号的那些字符，一律未作处理。再有，假设栈容量足够大，因此入栈时未判断溢出。

3. (1) A 和 D 是合法序列，B 和 C 是非法序列。
- (2) 设被判定的操作序列已存入一维数组 A 中。

```

int Judge(char A[])
//判断字符数组 A 中的输入输出序列是否是合法序列。如是，返回 true，否则返回 false。
{
i=0; //i 为下标。
j=k=0; //j 和 k 分别为 I 和字母 O 的个数。
while(A[i]!='\0') //当未到字符数组尾就作。
{
switch(A[i])
{
case'I': j++; break; //入栈次数增 1。
case'O': k++; if(k>j){printf("序列非法\n"); exit(0);} }
i++; //不论 A[i]是'I'或'O'，指针 i 均后移。}
if(j!=k) {printf("序列非法\n"); return(false);}
else {printf("序列合法\n"); return(true);} } //算法结束。
}

```

[算法讨论]在入栈出栈序列（即由'I'和'O'组成的字符串）的任一位置，入栈次数（'I'的个数）都必须大于等于出栈次数（即'O'的个数），否则视作非法序列，立即给出信息，退出算法。整个序列（即读到字符数组中字符串的结束标记'\0'），入栈次数必须等于出栈次数（题目中要求栈的初态和终态都为空），否则视为非法序列。

```

4.int MaxValue (int a[],int n)
//设整数序列存于数组 a 中，共有 n 个，本算法求解其最大值。
{
if (n==1) max=a[1];
else if a[n]>MaxValue(a,n-1) max=a[n];
else max=MaxValue(a,n-1);
return(max); }

```

5. [题目分析]从集合（1..n）中选出 k（本题中 k=2）个元素，为了避免重复和漏选，可分别求出包括 1 和不包括 1 的所有组合。即包括 1 时，求出集合（2..n）中取出 k-1 个元素的所有组合；不包括 1 时，求出集合（2..n）中取出 k 个元素的所有组合。将这两种情况合到一起，就是题目的解。

```

int A[],n; //设集合已存于数组 A 中。
void comb(int P[],int i,int k)
//从集合（1..n）中选取 k（k<=n）个元素的所有组合
{
if (k==0) printf(P);
else if(k<=n) {P[i]=A[i]; comb(P,i+1,k-1); comb(P,i+1,k); }
} //算法结束

```